

COMPUTER SCIENCE — Unit C1: Programming Fundamentals

Sessions 1–3 (6 hours total) | Track: Basics of Programming

Language used: Python (clean syntax, ideal for teaching concepts before moving to C/C++ per the syllabus)

Learning Objectives

Understand variables, data types, and operators.

Write and reason about conditional (if/else) logic.

Write and reason about loops (for, while).

Read simple input, produce output, and trace program execution by hand.

1. What is a variable?

A variable is a named location in memory that stores a value. In Python:

```
python
age = 20      # integer (int)
price = 19.99 # floating point (float)
name = "Alice" # string (str)
is_student = True # boolean (bool)
```

Key idea: the variable name is a *label*, the value is *data*. Assignment (`=`) copies the value on the right into the variable on the left.

Basic operators

Category	Operators	Example	Result
Arithmetic	+ − * // % **	7 // 2	3 (integer division)
		7 % 2	1 (remainder / modulo)
		2 ** 3	8 (exponent)
Comparison	== != < > <= >=	5 == 5	True
Logical	and, or, not	True and False	False

2. Conditional logic (branching)

```
python
```

```
temperature = 15

if temperature > 25:
    print("It's hot")
elif temperature > 10:
    print("It's mild")
else:
    print("It's cold")
```

Execution trace: Python checks conditions **top to bottom**, executes the **first true branch only**, then skips the rest.

temperature = 15 → `temperature > 25` is False → `temperature > 10` is True → prints "It's mild" → skips else.

3. Loops (repetition)

for loop — when you know how many times to repeat

```
python
for i in range(5): # i takes values 0, 1, 2, 3, 4
    print(i)
```

`range(5)` generates 0 through 4 (5 numbers, **stops before 5**).

while loop — repeat while a condition holds

```
python
count = 0
while count < 3:
    print("Count is", count)
    count = count + 1
```

Trace table (a critical skill — do this by hand before running code):

Iteration **count (before)** **condition (count < 3)** **printed** **count (after)**

1	0	True	"Count is 0"	1
2	1	True	"Count is 1"	2
3	2	True	"Count is 2"	3
4	3	False	— loop ends —	

4. Putting it together — a worked mini-program

Problem: Sum all even numbers from 1 to 20.

```
python
```

```
total = 0
for n in range(1, 21): # 1 through 20 inclusive
    if n % 2 == 0:      # n % 2 == 0 means n is even
        total = total + n

print("Sum of evens:", total)
```

Trace (partial):

n=1: $1\%2=1 \rightarrow$ skip

n=2: $2\%2=0 \rightarrow$ total = $0+2 = 2$

n=3: skip

n=4: total = $2+4 = 6$

... continues ...

Final total = $2+4+6+\dots+20 = 110$

5. Reading input

```
python
name = input("What is your name? ")
age = int(input("What is your age? ")) # input() always returns a string, so we convert
print(name, "will be", age + 1, "next year")
```

Key idea: `input()` always returns a **string**. If you need to do math with it, you must convert using `int()` or `float()`.

6. In-Session Guided Practice

Work these on paper/whiteboard first (trace by hand), then verify by running the code.

What does this print?

```
python
x = 10
y = 3
print(x // y, x % y, x ** 2)
```

Trace this loop and write down every value printed:

```
python
for i in range(2, 10, 2):
    print(i)
```

Fix the bug: this should print "Pass" if score ≥ 60 , else "Fail":

```
python
```

```
score = 55
if score > 60:
    print("Pass")
if score < 60:
    print("Fail")
```

Write a program that asks the user for a number and prints whether it's positive, negative, or zero.

Write a program that prints the multiplication table of 7 (7×1 through 7×10) using a for loop.

7. Take-Home Questions

Q1. Trace this code by hand (write the full trace table) and state the final printed output:

```
python
total = 1
for i in range(1, 6):
    total = total * i
print(total)
```

Q2. Write a program that counts how many numbers between 1 and 100 are divisible by both 3 and 5. (Hint: use `%` and `and`)

Q3. What is the difference between `for` and `while` loops? Give one example of a problem better suited to each.

Q4. Explain what happens if you write `while True:` with no `break` statement inside. Why is this dangerous?

Q5. Write a program that reads a user's age and prints "Minor" if under 18, "Adult" if 18–64, and "Senior" if 65 or older.

Answer Key (for tutor reference)

`x // y` = 10//3 = 3; `x % y` = 10%3 = 1; `x**2` = 100 → prints: `3 1 100`

`range(2,10,2)` → 2, 4, 6, 8 (stops before 10, step 2)

Bug: uses two separate `if` (both could theoretically run, and boundary score==60 prints neither). Fix with `if/else`:

```
python
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

4/5. (Practice, open-ended — verify logic manually with the student.)

Take-home: Q1. total = 1×1×2×3×4×5 = **120** (this is 5 factorial, 5!) Q2.

```
python
```

```
count = 0
for n in range(1, 101):
    if n % 3 == 0 and n % 5 == 0:
        count += 1
print(count) # answer: 6 (15,30,45,60,75,90)
```

Q3. `for` is best when the number of repetitions is known in advance (e.g., "do this 10 times"); `while` is best when repetition depends on a condition that isn't a fixed count (e.g., "keep asking until the user enters a valid password"). Q4. It creates an **infinite loop** — the program never stops because the condition `True` is always true and nothing changes it. Dangerous because it can freeze the program or consume resources indefinitely; a `break` or a condition that becomes False is required to exit. Q5.

```
☐
python
age = int(input("Enter age: "))
if age < 18:
    print("Minor")
elif age < 65:
    print("Adult")
else:
    print("Senior")
```